00 - Exam Pattern Analysis (2021 - 2024 + Practice Task 3)

What the Python lab exams reward, how they have changed, and what to expect in next week's assessment. Read this first (10 minutes), then follow the study plan in section 8. ***Version 2: now built around the instructor's official checklist (section 0).***

0. The instructor's official checklist (read this before anything else)

Your instructor says the practical assessment (15% of the course) covers *the same main areas as the final-exam Python question*, and that you should **practise the individual operations and understand each line - not memorise one complete solution**. These are the exact bullets, and where the pack covers each:

Area	Checklist bullet	Where to practise
Pandas	loading CSV files	T5, T9 card
Pandas	checking the shape of a DataFrame	T5, T9
Pandas	identifying and removing duplicates	T5, T9
Pandas	**merging DataFrames using a common column**	T5 (new section), T9, Cheat Sheet B
Pandas	selecting and filtering data	T5, T9
Pandas	saving a DataFrame to a CSV file	T5, T9
Matplotlib	scatter, line, bar and **box** plots	T6 (box plot is a new section), T9, Cheat Sheet C
Matplotlib	titles and axis labels	T6, every template
SQLite	create / connect to a database	T4, T9
SQLite	**store a DataFrame in a table** (`to_sql`)	T4 (new section), Cheat Sheet D
SQLite	SELECT, WHERE, ORDER BY and **UPDATE**	T4 (UPDATE is a new worked section)
SQLite	parameterised queries with `?`	T4
SQLite	committing changes	T4
SQLite	**retrieve results and export them to CSV**	T4 (new section), Cheat Sheet D
PyTorch	**random seed before generating data**	T7 (new regression section), Cheat Sheet F
PyTorch	`nn.Module`, Linear + ReLU	T7
PyTorch	**synthetic data** `y = X @ w + b + noise` (continuous target)	T7 (new), Cheat Sheet F
PyTorch	shapes of X and y	T7 (new), Cheat Sheet F
PyTorch	choose the loss function and optimiser	T7
PyTorch	training loop; record and plot the loss	T7, P1, Cheat Sheet F

What this changes (compared with version 1 of this pack): 1. The PyTorch task is a **regression** task (continuous target, `MSELoss`, **no sigmoid** on the output). The 2024 paper's sigmoid network is still good practice but is not the template to expect. 2. **Merge, box plot, UPDATE,** `to_sql` **and CSV export** are now explicitly on the list; they were thin or missing before. 3. Practise **small operations**, each from a blank line. Do not rely on retyping a whole paper solution. 4. The seed goes **before** the data *and before* the model is built (weights are random too).

1. The five papers side by side

	2021	2022	2023	2024	Practice Task 3
Python marks	24 + 1 style	25	25	**40**	(practice for task 3)

	2021	2022	2023	2024	Practice Task 3
Tasks	Registration form	Bus-fare calculator	Library + "ChatGPT"	Sales data: 3 tasks	Height logger
tkinter GUI	yes (4 marks)	yes (4 marks)	yes (5 marks)	**no**	no
SQLite database	yes, `Users`	yes, `TravelPrice`	yes, `LibraryDB`	yes, `SalesDB`	no (CSV file)
"No duplicates" rule	usernames	-	ISBNs	rows	no re-create of CSV
Input validation	username, password	(age)	ISBN, year, empties	missing values, Total	numeric height, `q`
pandas / CSV	-	-	-	**yes**	**yes**
matplotlib	-	-	-	**yes (3 plots)**	**yes (histogram)**
PyTorch	-	-	-	**yes (12 marks)**	**replaced** by CSV/numpy/plot
Open book	not stated	not stated	not stated	**yes**	-
One `.py` file, `your_name_python_task.py`	yes	yes	yes	yes	-

Reading the trend: 2021-2023 = *small apps* (screen + database + validation). 2024 = *data pipeline* (CSV -> clean -> chart -> database -> model). The course material has since **dropped tkinter** and the practice sheet shows **PyTorch being swapped for CSV + pandas + numpy + matplotlib**. So the mock will almost certainly be a data-pipeline paper, with the old "validation + database" skills hiding inside it.

2. Marks by skill (counted from the marks printed on each paper)

Each column adds up to the paper's itemised marks (2021-2023 papers are "out of 25": 24 itemised + 1 for style/overall).

Category	2021	2022	2023	2024
GUI screen layout	4	4	5	0
Create DB + save into it	4	7	4	9
Query / display from the DB (incl. the "librarian")	3	11	10	6
Validation (duplicates, rules, messages)	11	0	3	0
"Clear all"	2	2	2	0
pandas cleaning + charts	0	0	0	13
Neural network	0	0	0	12
Itemised total	**24**	**24**	**24**	**40**

Database-related marks (create + save + query/display): **7/24, 18/24, 14/24, 15/40** = 29%, 75%, 58%, 38%.

Takeaway: SQLite is the one topic in *every* paper, never under 29% of the marks. Validation ("do something sensible when the data is wrong") is the second constant. In 2024 the screen disappeared and pandas + charts + a model took its place. Partial credit is generous (1-4 marks per sub-task), so a working answer to every sub-task beats a perfect answer to half.

3. How the questions are worded (decode them)

Verb in the question	What the marker wants to SEE
"Load the data"	`pd.read_csv(...)` and a printed `head()`
"Ensure there are no missing values"	evidence: `isna().sum()` before/after + a `fillna` for **each** column that has blanks

Verb in the question	What the marker wants to SEE
"Convert ... to a datetime object"	`pd.to_datetime(...)` (and the right day/month order)
"Ensure Total correctly represents..."	recompute `Total = Quantity * Price`
"Create a bar chart / line plot"	the chart **plus** title and axis labels
"Create an SQLite database named X"	`sqlite3.connect("X.db")`, exactly that name
"Create a table named Sales with appropriate columns"	`CREATE TABLE`, sensible types
"Ensure no duplicates are added"	a UNIQUE/PRIMARY KEY rule **or** a check before insert
"Write a query to ..."	an SQL string run with `execute`, and the answer **printed**
"Display in descending order"	`ORDER BY ... DESC`
"Record the loss for each epoch"	a list + `append(loss.item())`
"Display a suitable message"	`print(...)` (console) or `messagebox` (GUI) - plain English that names the problem

Hidden mark patterns across all papers 1. **Names are marks.** The database name is worth a mark on its own (2022: "called TravelPrice [1 mark]"). Spell it exactly: `TravelPrice`, `LibraryDB`, `Users`, `SalesDB`; table `Sales`. 2. **Every sub-task is 1-4 marks:** partial credit is generous. A half-working answer to each sub-task beats a perfect answer to half of them. 3. **Tasks build on each other** (2024: Task 2 stores "the data processed in Part 1"). A broken Task 1 hurts Task 2. Get Task 1 *running* quickly. 4. **Data has planted problems:** blanks, duplicates, mixed years, `dd/mm/yyyy` dates, unknown product names. The 2024 `sales_data.csv` has 6 blank products, 1 blank price, 3 blank totals, 6 exact duplicate rows and 3 different years. Expect the mock's CSV to have similar traps. 5. **Output is checked by running your file.** Use `print()` generously: a correct but silent query earns nothing. 6. **Style marks** (2021: 1 mark; Java parts: clean code): comments, good names, tidy layout.

4. Likelihood forecast for the mock (my estimates from the trend, not facts)

Topic	Chance it appears	Where to revise
pandas: load, shape, duplicates, **merge**, filter, save to CSV	~95% (all on the instructor's list)	T5, T9
matplotlib: scatter, line, bar, **box**, with titles/labels	~95% (on the list)	T6, T9
SQLite: connect, `to_sql`, SELECT/WHERE/ORDER BY, **UPDATE**, `?`, commit, **export to CSV**	~95% (on the list)	T4, T9
PyTorch **regression**: seed, `y = X @ w + b + noise`, shapes, Linear+ReLU, MSE, loop, plot loss	~90% (on the list)	T7, T9, Cheat Sheet F
`fillna`, `to_datetime`, `strftime` year filtering (2024 style)	~50%	T5, P1
"No duplicates" in a database (`UNIQUE`, `INSERT OR IGNORE`)	~50%	T4, P1
CSV file handling, append vs create, `q` to quit loop	~30%	T3
sigmoid / BCELoss classification network (2024 style)	~20%	T7, P1
OOP (class, `__init__`, `__str__`)	~15% (Java side carries OOP)	T2
tkinter	~2% (removed)	T8, skip
Console menus, functions, validation loops	glue in everything	T1

Probabilities are my judgement, now weighted heavily towards the instructor's checklist: anything on that list should be treated as certain to be examined.

5. The ten commandments for exam day

1. **Read the whole paper once (3 min)**, then underline every noun that must be spelled exactly (file, DB, table, column names).

2. **Start the file with all imports** and a comment header: name, date, task list.
3. **Do Task 1 first and RUN it** before moving on. Run after every sub-task. Never write 100 lines blind.
4. **Look at the data** (`head`, `isna().sum()`, `duplicated().sum()`, `dtypes`) before cleaning.
5. **Label every section with a comment copied from the question**, e.g. `# Task 2b: Total sales for 2023`.
6. **Print every answer** with a label.
7. **Assign back** (`df["c"] = df["c"].fillna(...)`); never rely on `inplace=True` for a single column.
8. **Use `?` placeholders** in SQL, `commit()` after writes, `close()` at the end.
9. **Make the script re-runnable**: `IF NOT EXISTS`, `UNIQUE`, `INSERT OR IGNORE`, delete the `.db` / `.csv` while testing if you must.
10. **Last 15 minutes:** run the full file from a clean start (delete the `.db`), check file name, check every title/label, save, upload. Do not start anything new.

6. Mistakes that cost marks (from the five papers and the official solution)

Mistake	Effect	Cure
`fillna(0)` on Price before recomputing Total	Total becomes 0, sales under-reported (the official solution does this: £8,900 instead of £8,950)	recover Price = Total / Quantity first
`INSERT OR IGNORE` with no UNIQUE rule	duplicates inserted; rerun doubles the table (151 -> 302 rows)	`UNIQUE(...)` + `drop_duplicates()`
Line chart over all years when asked for 2023	wrong chart	filter `df["Date"].dt.year == 2023` first
Forgetting `commit()`	nothing saved	commit after inserts
Date left as text	cannot filter by year/month	`pd.to_datetime(..., dayfirst=True)`
numpy ints/Timestamps into SQLite	stored as BLOB junk or an error	convert: `int(x)`, `float(x)`, `dt.strftime(...)`
`command=func()` in tkinter	runs at start-up	`command=func`
Two charts without `plt.figure()`	second drawn on top of first	new `plt.figure()` per chart
Wrong layer sizes in `nn.Linear`	shape error	out of one = in of next
Missing `zero_grad()`	gradients accumulate silently	five-step loop chant
Not testing the full file at the end	crash on line 140	run top-to-bottom before upload

7. Time budget for a 2-hour mock (adjust to the real length)

Minutes	Do
0-5	read paper, list exact names, look at the data
5-40	Task 1 (pandas, cleaning, charts) - run it
40-80	Task 2 (SQLite) - run it, run it twice to test duplicates
80-105	Task 3 (network / CSV logger) - run it
105-120	clean comments, rerun from scratch, filename, submit

If a task is stuck for 10 minutes, write a comment saying what you would do, leave a stub that runs, and **move on**. Come back with spare time.

8. Your study plan (version 2: operations first, then whole programs)

Principle (from your instructor): practise individual operations, understand each line, do not memorise one solution.

Order	Study	Documents	Practise
1	**The operation cards**: 27 small operations, one per checklist bullet	T9 + `Solutions_py/practical_operation_drills.py`	each day: cover the code, write 5 cards from memory, run them, tick the tracker
2	pandas (merge, duplicates, filter, save)	T5	merge two small CSVs of your own invention; compare `shape` before/after
3	matplotlib (scatter, line, bar, box)	T6	4 plots from one DataFrame, each with title + labels, 10 minutes
4	SQLite (to_sql, SELECT/UPDATE with `?`, export CSV)	T4	create DB -> store DataFrame -> update -> export, from a blank file
5	PyTorch regression	T7 + `Solutions_py/pytorch_regression_practical.py`	seed -> data -> model -> loop -> plot, change one thing at a time (N, F, lr, epochs)
6	**Mock C** (written to the checklist), timed	Mock_Exams/MockC	mark yourself with the scheme
7	Mock A and B (more whole-program practice)	Mock_Exams	optional but good
8	The 2024 paper explained	P1	**read for understanding** (what and why of each line); do NOT try to memorise the file
9	Python core + OOP + files/CSV gaps	T1, T2, T3	only what you got wrong
last	Hand-write the Quick-Reference boxes; print the cheat sheet	Exam_Day_Cheat_Sheet (v2)	the sheet you carry in

Printing advice: see START_HERE (version 2) for the exact list.